

Rapport présenté pour la validation du module

Développement mobile

Application mobile pour projet Scankool

Année universitaire

2025 – 2026

Établissement

Ecole marocaine des sciences de l'ingénieure

Filière

Ingénierie informatique et réseau

Réalisé par :

Hamza JAADA

Encadré par :

Mme. Naji Zineb

Sommaire

Sommaire	2
Remerciements	3
Introduction Générale.....	4
Chapitre 1 : Contexte du Projet	5
1.1 Présentation générale du projet	5
1.2 Problématique.....	5
1.3 Objectifs du projet	6
1.4 Périmètre fonctionnel	7
Chapitre 2 : Conception	8
2.1 Analyse fonctionnelle	8
2.2 Identification des acteurs.....	9
2.3 Description du processus global	9
2.4 Conception technique.....	10
2.6 Choix technologiques.....	10
2.7 Sécurité et fiabilité	12
Chapitre 3 : Réalisation	14
3.1 Structure du projet	14
3.1.1 Organisation du dossier src/	15
3.1.2 Justification de l'architecture	16
3.2 Intégration avec l'API Django	17
3.3 Tests et validation	19
Conclusion Générale	19

Remerciements

Je tiens tout d'abord à exprimer ma profonde gratitude à **Mme NAJI Zineb** notre enseignante en Développement Mobile, pour son encadrement précieux, sa disponibilité constante et ses conseils avisés tout au long de ce projet. Son expertise technique dans le domaine du développement mobile avec React Native, sa pédagogie claire et son soutien personnalisé ont été déterminants pour la réussite de ce travail. Ses retours constructifs lors des revues de code et ses orientations stratégiques ont grandement contribué à l'amélioration continue du projet.

Je remercie chaleureusement **l'ensemble de l'équipe pédagogique** du module de Développement Mobile React Native pour la qualité exceptionnelle des enseignements dispensés. La progression pédagogique bien structurée, allant des bases fondamentales aux concepts avancés, m'a permis d'acquérir les compétences techniques nécessaires à la conception et au développement de cette application mobile complète. Les supports de cours riches et les travaux pratiques pertinents ont constitué une base solide pour la réalisation de ce projet.

Un remerciement tout particulier s'adresse à **mes collègues de promotion** pour les échanges fructueux, l'entraide mutuelle et l'ambiance collaborative qui ont caractérisé cette période de développement. Les discussions techniques enrichissantes, le partage de ressources et la solidarité face aux défis rencontrés ont largement contribué à l'enrichissement de ce projet et ont renforcé mes compétences en travail d'équipe.

Je tiens également à remercier **l'administration de Ecole marocaine des science de l'ingénieure** pour avoir mis à disposition les ressources pédagogiques et les infrastructures techniques nécessaires à la réalisation de ce travail. L'environnement de travail favorable et les outils mis à disposition ont grandement facilité le processus de développement.

Introduction Générale

Ce rapport présente mon projet de fin de module en Développement Mobile avec React Native. Dans le cadre de cette validation, nous devons créer une application mobile complète avec frontend et backend.

J'ai choisi de développer **Scankool Mobile**, une application de gestion de menus pour restaurateurs. C'est la version mobile d'un projet web que j'avais déjà réalisé. L'application permet aux gérants de restaurants de gérer leurs menus depuis leur smartphone : créer des comptes, modifier des prix, ajouter des photos, organiser les plats par catégories, etc.

L'idée m'est venue en observant que les restaurateurs sont souvent loin de leur ordinateur mais toujours avec leur téléphone. Ils ont besoin de modifier rapidement leurs menus selon les disponibilités ou les prix du marché. Scankool Mobile répond à ce besoin pratique.

Pour ce projet, j'utilise la même API que j'avais développée avec Django pour la version web. Cela montre comment on peut créer une application mobile qui fonctionne avec un backend existant.

Dans ce rapport, je vais vous expliquer :

- Comment j'ai analysé les besoins des utilisateurs
- Les choix techniques que j'ai faits pour le développement
- Comment j'ai construit l'application étape par étape
- Les difficultés rencontrées et comment je les ai résolues

C'est un projet concret qui met en pratique tout ce que j'ai appris sur React Native, tout en répondant à un vrai besoin professionnel dans le secteur de la restauration.

Chapitre 1 : Contexte du Projet

1.1 Présentation générale du projet

Scankool Mobile est une application mobile développée avec React Native qui permet aux restaurateurs de gérer leurs menus de manière simple et intuitive. Conçue spécifiquement pour les professionnels de la restauration, cette application répond au besoin croissant de gestion mobile dans un secteur où la réactivité et la flexibilité sont essentielles.

L'application se connecte à une API Django existante, déjà utilisée par la version web du projet. Cette architecture garantit une parfaite synchronisation des données entre les différentes plateformes et permet aux restaurateurs de passer sans effort de l'ordinateur au smartphone selon leurs besoins du moment. Que ce soit pour modifier un prix rapidement en cuisine, ajouter un plat spécial depuis le marché, ou simplement consulter son menu pendant les heures de service, Scankool Mobile offre la mobilité nécessaire.

Le projet s'inscrit dans une démarche de digitalisation du secteur de la restauration, où les outils traditionnels (cahiers, fichiers Excel) montrent leurs limites face à la nécessité de mises à jour fréquentes et de présentation professionnelle. En proposant une solution mobile connectée, Scankool Mobile modernise la gestion des menus tout en la rendant plus accessible et efficiente.

1.2 Problématique

Les restaurateurs passent une grande partie de leur temps en déplacement ou en salle, loin de leur ordinateur. La gestion des menus depuis un ordinateur fixe présente plusieurs limitations :

- Impossibilité de modifier rapidement les prix en fonction du marché
- Difficulté à mettre à jour les disponibilités en temps réel
- Absence de notification immédiate des changements
- Dépendance à un poste de travail fixe

Problématique principale : Comment permettre aux restaurateurs de gérer efficacement leurs menus en temps réel depuis leur mobile, tout en garantissant la synchronisation avec une plateforme web existante ?

1.3 Objectifs du projet

Objectif principal :

Créer une application mobile avec React Native qui aide vraiment les restaurateurs à gérer leurs menus. L'application doit être simple à utiliser sur téléphone, même pour ceux qui ne sont pas très à l'aise avec la technologie.

Objectifs spécifiques :

1. **Sécurité des comptes :** Mettre en place un système de connexion sécurisé pour protéger les données des restaurants.
2. **Gestion complète des menus :** Permettre aux restaurateurs de faire toutes les opérations de base : créer, voir, modifier et supprimer leurs menus.
3. **Organisation facile :** Aider à organiser les plats par catégories (entrées, plats principaux, desserts...) et permettre de les modifier simplement.
4. **Gestion des photos :** Pouvoir prendre ou choisir des photos des plats directement avec le téléphone, et les ajouter facilement au menu.
5. **Connexion à l'API existante :** Utiliser la même API que la version web, sans avoir à tout modifier côté serveur.
6. **Interface adaptée au mobile :** Créer une interface spécialement pensée pour le téléphone, avec des gros boutons, une navigation simple et un chargement rapide.
7. **Synchronisation en temps réel :** Faire en sorte que les changements sur le téléphone soient immédiatement visibles sur le site web, et inversement.

L'idée, c'est de créer un outil pratique qui résout un vrai problème : les restaurateurs ont besoin de modifier leurs menus rapidement, souvent loin de leur ordinateur, et veulent que tout soit à jour partout en même temps.

1.4 Périmètre fonctionnel

Fonctionnalités incluses :

1. **Inscription et connexion** : Les restaurateurs peuvent créer un compte et se connecter à l'application.
2. **Tableau de bord** : Une page d'accueil qui montre un résumé de leurs menus et statistiques simples.
3. **Gestion des menus** : Créer plusieurs menus, les modifier ou les supprimer.
4. **Organisation par catégories** : Classer les plats dans des catégories comme entrées, plats principaux, desserts.
5. **Modification des plats** : Changer les prix, les descriptions et les détails des plats.
6. **Photos des plats** : Ajouter, modifier ou supprimer des photos pour chaque plat.
7. **Mises à jour en direct** : Voir immédiatement les changements effectués.

Chapitre 2 : Conception

2.1 Analyse fonctionnelle

L'analyse fonctionnelle a permis d'identifier les besoins essentiels des utilisateurs et de structurer l'application en modules cohérents :

Module d'authentification :

- Connexion avec email/mot de passe
- Inscription avec informations du restaurant
- Récupération de mot de passe
- Gestion de session (JWT tokens)

Module de gestion des menus :

- Création d'un nouveau menu
- Édition des informations du menu

Module de gestion des catégories :

- Ajout de catégories (Entrées, Plats principaux, Desserts, etc.)
- Réorganisation des catégories
- Modification des noms de catégories
- Suppression de catégories (avec gestion des plats associés)

Module de gestion des plats :

- Ajout de nouveaux plats
- Modification des informations (nom, description, prix)
- Upload de photos
- Changement de catégorie
- Mise en avant de plats spéciaux

Module dashboard :

- Vue d'ensemble des statistiques
- Accès rapide aux menus
- Notifications des modifications récentes
- Informations sur le restaurant

2.2 Identification des acteurs

Acteur principal : Le Restaurateur (Gérant)

- Crée et gère son compte
- Administre ses menus
- Modifie les prix et disponibilités
- Télécharge des photos de plats

Acteur secondaire : L'Administrateur Système

- Gère les comptes utilisateurs
- Supervise le bon fonctionnement de l'API
- Assure la maintenance technique

Système externe : API Django

- Fournit les services backend
- Gère l'authentification
- Stocke les données en base
- Traite les uploads de fichiers

2.3 Description du processus global

Le processus global peut être décrit en plusieurs étapes :

1. Phase d'onboarding :

- Le restaurateur télécharge l'application
- Crée un compte ou se connecte
- Configure son restaurant (nom, adresse, type de cuisine)

2. Phase de configuration initiale :

- Création du premier menu
- Ajout des catégories de base
- Import ou création des premiers plats

3. Phase d'utilisation quotidienne :

- Consultation du dashboard
- Modification des prix selon le marché
- Ajout de nouveaux plats
- Mise à jour des photos
- Gestion des disponibilités

4. Phase de maintenance :

- Réorganisation des menus
- Archivage des plats saisonniers
- Mise à jour des informations du restaurant

2.4 Conception technique

L'architecture technique repose sur une séparation claire entre :

- Le frontend mobile (React Native)
- Le backend API (Django REST Framework)
- La base de données (PostgreSQL/MySQL)
- Le stockage des fichiers (AWS S3 ou équivalent)

2.6 Choix technologiques

Frontend Mobile :



- **React Native 0.72:** Framework cross-platform qui permet de développer une application mobile pour iOS et Android en utilisant le même code JavaScript. Cela réduit considérablement le temps de développement car on n'a pas besoin de créer deux applications séparées. React Native utilise des composants natifs, ce qui donne une expérience utilisateur fluide et performante.

Backend (déjà existant) :

django

- **Django 4.2 :** Framework web Python robuste et sécurisé qui fournit déjà l'API REST complète pour le projet. Django offre une structure organisée avec des fonctionnalités intégrées pour l'authentification, la gestion des bases de données et la sécurité. Comme l'API existe déjà pour la version web, cela évite de devoir recréer un backend spécifique pour l'application mobile.



- **MySQL :** Base de données relationnelle performante qui stocke toutes les données du projet (comptes utilisateurs, menus, plats, catégories, photos). MySQL est fiable, bien documenté et largement utilisé dans l'industrie, ce qui garantit la stabilité et la pérennité des données.
- **Outils utilisés :**



- **Visual Studio Code :** Éditeur de code léger mais puissant avec un excellent support pour React Native. Ses extensions facilitent le débogage, la coloration syntaxique et la gestion du projet. L'IntelliSense (suggestions de code) aide à écrire du code plus rapidement et avec moins d'erreurs.



- **Postman** : Outil essentiel pour tester les endpoints de l'API Django. Il permet de vérifier que les requêtes HTTP fonctionnent correctement, de simuler différents scénarios d'utilisation et de documenter les échanges entre le mobile et le serveur.



- **Git** : Système de contrôle de version qui permet de sauvegarder l'historique des modifications du code. Git facilite le travail en équipe, la gestion des différentes versions de l'application et le retour en arrière en cas de problème.

2.7 Sécurité et fiabilité

1. Authentification :

- Tokens JWT avec expiration courte (15 minutes)
- Refresh tokens sécurisés
- Stockage sécurisé avec AsyncStorage
- Logout automatique après inactivité

2. Protection des données :

- HTTPS obligatoire pour toutes les communications
- Validation des données côté client et serveur
- Sanitization des inputs utilisateur
- Protection contre les injections SQL (gérée par Django ORM)

3. Sécurité des fichiers :

- Validation du type et taille des images
- Renommage aléatoire des fichiers uploadés
- Stockage sécurisé sur serveur
- Limitations de taux d'upload

4. Fiabilité du système :

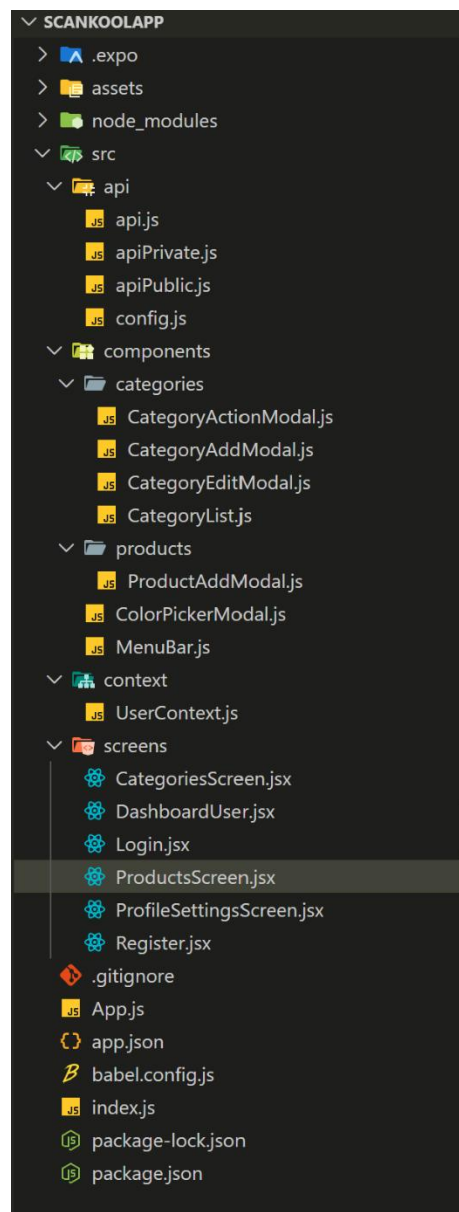
- Retry automatique sur échec réseau
- Cache stratégique pour les données fréquemment consultées
- Messages d'erreur utilisateur-friendly
- Sauvegarde automatique des formulaires longs

Chapitre 3 : Réalisation

3.1 Structure du projet

L'application Scankool Mobile est organisée selon une architecture modulaire claire qui facilite la maintenance et l'évolution du code. Voici la structure détaillée des principaux dossiers et fichiers :

Structure principale :



3.1.1 Organisation du dossier src/

Dossier api/ - Gestion des communications avec le backend :

- api.js : Configuration principale d'Axios et intercepteurs
- apiPrivate.js : Requêtes authentifiées (avec token JWT)
- apiPublic.js : Requêtes publiques (connexion, inscription)
- config.js : Configuration des URLs et constantes API

Dossier components/ - Composants réutilisables :

Sous-dossier categories/ :

- CategoryActionModal.js : Modal d'actions pour une catégorie (modifier/supprimer)
- CategoryAddModal.js : Formulaire d'ajout d'une nouvelle catégorie
- CategoryEditModal.js : Formulaire de modification d'une catégorie existante
- CategoryList.js : Composant d'affichage de la liste des catégories

Sous-dossier products/ :

- ProductAddModal.js : Formulaire d'ajout d'un nouveau plat
- ColorPickerModal.js : Sélecteur de couleurs pour personnalisation

Composants communs :

- MenuBar.js : Barre de navigation latérale ou inférieure

Dossier context/ - Gestion d'état global :

- UserContext.js : Contexte React pour gérer les données utilisateur et l'authentification

Dossier screens/ - Écrans principaux de l'application :

- CategoriesScreen.jsx : Gestion des catégories de plats
- DashboardUser.jsx : Tableau de bord utilisateur avec statistiques
- Login.jsx : Écran de connexion
- ProductsScreen.jsx : Gestion des plats
- ProfileSettingsScreen.jsx : Paramètres du profil utilisateur
- Register.jsx : Écran d'inscription

3.1.2 Justification de l'architecture

Cette structure présente plusieurs avantages :

1. **Séparation des préoccupations** : Chaque dossier a une responsabilité claire
2. **Réutilisabilité** : Les composants sont isolés et peuvent être utilisés dans plusieurs écrans
3. **Maintenabilité** : La logique métier est séparée de l'interface utilisateur
4. **Évolutivité** : Facile d'ajouter de nouvelles fonctionnalités sans désorganiser le code existant

L'utilisation des Contexts React (UserContext.js) permet de gérer l'état de l'utilisateur (connexion, données) de manière efficace sans prop drilling. Les modals sont séparés pour une meilleure isolation de la logique de formulaire.

Cette organisation reflète les bonnes pratiques du développement React Native et facilite la collaboration ainsi que la maintenance à long terme du projet.

3.2 Intégration avec l'API Django

L'intégration avec l'API Django existante a été réalisée avec une architecture en trois couches pour assurer fiabilité et sécurité.

Configuration des services API :

Services publics (apiPublic.js) :

- Gestion des endpoints accessibles sans authentification
- Connexion et inscription des utilisateurs
- Récupération des informations publiques des restaurants
- Validation des tokens sans session active

Services privés (apiPrivate.js) :

- Tous les endpoints nécessitant une authentification JWT
- Gestion des menus, catégories et plats
- Upload des fichiers et images
- Accès aux données utilisateur et statistiques

Configuration centrale (config.js) :

- Définition des URLs de base selon l'environnement (dev/prod)
- Timeouts configurables pour chaque type de requête
- Paramètres de retry en cas d'échec réseau

Gestion des tokens JWT :

Intercepteurs intelligents :

- Injection automatique du token d'accès dans les headers
- Détection des erreurs 401 (token expiré)
- Tentative automatique de rafraîchissement du token
- Redirection vers l'écran de login en cas d'échec d'authentification

Persistance sécurisée :

- Stockage chiffré des tokens via AsyncStorage
- Rotation régulière des tokens de rafraîchissement
- Nettoyage automatique des tokens expirés

Optimisation des communications :

Mise en cache stratégique :

- Cache en mémoire pour les données fréquemment consultées
- Validation par ETag pour éviter les transferts inutiles
- Politique de cache différenciée selon le type de données

Gestion des erreurs :

- Messages d'erreur utilisateur-friendly
- Logging détaillé pour le débogage
- Retry automatique sur les erreurs temporaires

Cette intégration assure une communication fluide et fiable entre l'application mobile et le backend Django, tout en maintenant une expérience utilisateur optimale même dans des conditions réseau difficiles.

3.3 Tests et validation

14:00
◀ Code Scanner



Créer un compte

AdminReact

React

react1235 

06521349812

Créer un compte →

[Déjà un compte ? Se connecter](#)

13:59
◀ Code Scanner



Welcome back!

Connectez-vous pour accéder à votre compte.

Entrez votre username

Entrez votre mot de passe 

Connexion

Pas encore inscrit ? [Inscrivez-vous !](#)

L'interface de Scankool Mobile est simple et facile à utiliser, spécialement conçue pour les restaurateurs qui sont souvent pressés.

Écran d'inscription :

Quand un nouveau restaurateur arrive, il voit un écran clair pour créer son compte. Il entre juste son nom d'utilisateur, son mot de passe et son numéro de téléphone. Un grand bouton "Créer un compte" lui montre clairement quoi faire.

Écran de connexion :

Pour les restaurateurs qui reviennent, l'écran dit "Welcome back!" pour les accueillir. Ils entrent juste leur nom d'utilisateur et mot de passe, puis cliquent sur "Connexion". C'est très simple.

Facile à comprendre :

- Les boutons sont gros et bien visibles
- Les champs à remplir sont clairs
- On sait toujours quoi faire ensuite
- Pas trop d'informations en même temps

L'idée, c'est que même un restaurateur très occupé puisse créer son compte ou se connecter en moins d'une minute, sans se poser de questions.

14:00



Welcome back!

Connectez-vous pour accéder à votre compte.



Connexion

Pas encore inscrit ? [Inscrivez-vous !](#)

14:01



Scan



0

Catégories totales

0

Produits totaux

0

Produits indisponibles

0

Catégories indisponibles

Produits indisponibles

Aucun produit indisponible

Catégories indisponibles

Aucune catégorie indisponible

Après la connexion, le restaurateur arrive sur son tableau de bord. C'est comme une page d'accueil qui lui montre tout ce qui est important.

Au centre :

Des cartes d'information qui résument l'état du restaurant :

1. "Catégories totales" : Combien de catégories de plats il a créées
2. "Produits indisponibles" : Combien de plats sont temporairement en rupture
3. "Catégories indisponibles" : Combien de catégories sont désactivées

En bas :

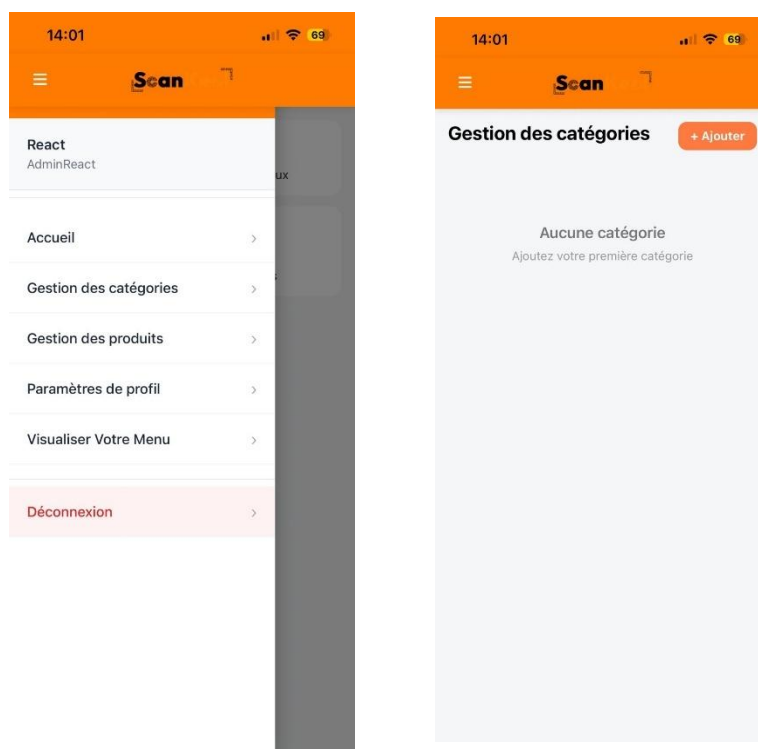
Si tout va bien, des messages rassurants comme :

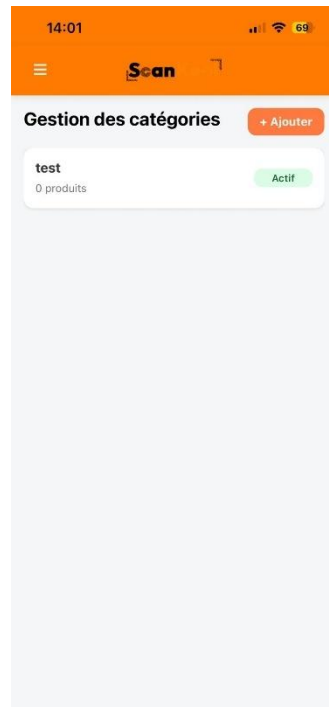
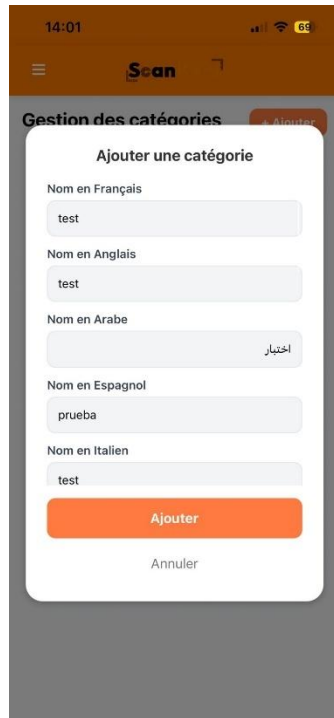
- "Aucun produit indisponible"
- "Aucune catégorie indisponible"

Pourquoi c'est pratique :

- Le restaurateur voit tout d'un coup d'œil
- Il repère vite les problèmes (plats manquants)
- Pas besoin de chercher dans les menus pour savoir l'état de son restaurant
- C'est clair et simple, même en pleine heure de service

C'est comme un tableau de bord de voiture qui montre l'essentiel : on voit tout ce qui est important sans être submergé d'informations.





Gestion des catégories :

Cette partie permet au restaurateur d'organiser ses catégories, comme "Entrées", "Plats principaux", "Desserts".

Pour ajouter une catégorie :

Un formulaire simple apparaît. Le restaurateur peut donner un nom à sa catégorie en plusieurs langues (français, anglais, arabe, espagnol, italien). Ça permet aux clients étrangers de mieux comprendre le menu. Il suffit d'écrire le nom dans chaque langue et de cliquer sur "Ajouter".

Liste des catégories :

En dessous, on voit toutes les catégories déjà créées. Pour chaque catégorie, on voit :

Son nom

Combien de plats elle contient (par exemple "0 produits")

Si elle est active ou non

C'est pratique parce que :

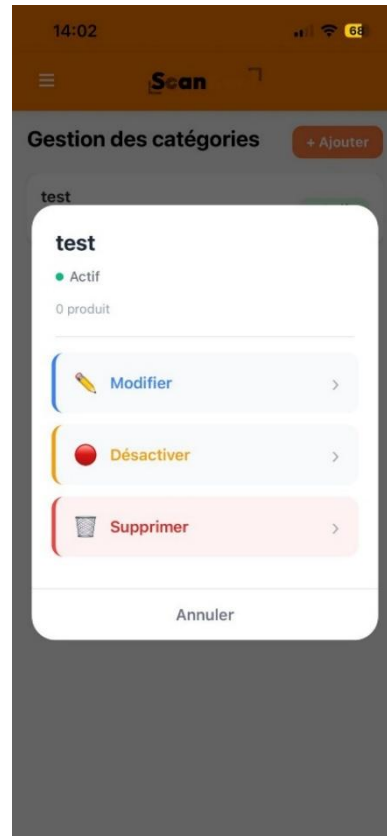
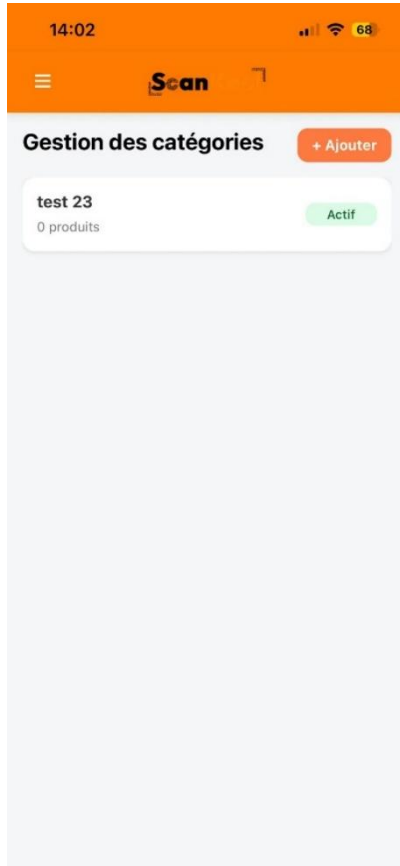
Le restaurateur peut organiser son menu comme il veut

Il voit tout de suite quelles catégories sont vides

Il peut activer ou désactiver des catégories selon la saison

L'ajout est très simple, même pour ajouter plusieurs langues

Ça aide à garder le menu bien organisé et à jour, surtout quand on a beaucoup de plats différents.



Liste des catégories :

Cette écran montre toutes les catégories que le restaurateur a créées. C'est présenté comme une liste déroulante qu'on peut faire défiler du doigt.

Pour chaque catégorie on voit :

1. Le nom de la catégorie (exemple : "test 23", "test 24", etc.)
2. Le nombre de produits dans cette catégorie (pour l'instant 0 partout dans l'exemple)
3. Le statut : "Actif" ou "Inactif"
4. Les actions possibles : Ajouter, Modifier, Désactiver, Supprimer

Navigation facile :

Même s'il y a beaucoup de catégories (jusqu'à 162 dans l'exemple), on peut facilement :

- Faire défiler pour voir toutes les catégories
- Trouver rapidement une catégorie spécifique
- Agir sur chaque catégorie individuellement avec les boutons appropriés

Pourquoi c'est pratique :

- Le restaurateur voit tout son organisation en un seul endroit
- Il sait quelles catégories sont vides (0 produits)
- Il peut activer/désactiver selon la saison ou les disponibilités
- La liste reste lisible et organisée même avec beaucoup d'entrées

C'est comme un répertoire de toutes les catégories de plats, toujours à portée de main pour gérer le menu facilement.

The screenshot shows the 'Modifier la catégorie' (Edit category) form within the 'Gestion des catégories' (Category Management) section of the Scan app. The form has a white background and is set against a dark brown header. It contains several input fields for the category name in different languages: 'Nom en Français *' (test 23), 'Nom en Anglais' (test 23), 'Nom en Arabe' (اختبار 23), 'Nom en Espagnol' (prueba 23), and 'Nom en Italien'. At the bottom of the form are two buttons: 'Enregistrer' (Save) in green and 'Annuler' (Cancel) in red. The status bar at the top shows the time as 14:02 and a 68% battery level.

The screenshot shows the 'Gestion des catégories' (Category Management) screen in the Scan app. It features an orange header with the 'Scan' logo. Below the header, there is a list of categories. The first visible entry is 'test 23' with '0 produits' (0 products) and an 'Inactif' (Inactive) status. A '+ Ajouter' (Add) button is located in the top right corner. The status bar at the top shows the time as 14:02 and a 68% battery level.

Modification d'une catégorie (écran de gauche)

- Lorsqu'on modifie une catégorie, une fenêtre s'ouvre par-dessus l'écran principal.
- Cette fenêtre contient un formulaire multilingue :
 - Nom en Français
 - Nom en Anglais
 - Nom en Arabe
 - Nom en Espagnol
 - Nom en Italien
- Les champs sont préremplis avec les valeurs existantes de la catégorie.

En bas de la fenêtre :

- Enregistrer : pour sauvegarder les modifications
- Annuler : pour fermer sans enregistrer

L'arrière-plan est grisé, ce qui montre que l'utilisateur est en mode édition.

Conclusion Générale